

GAME SERVER PORTFOLIO

MMORPG-with-IOCP

Windows IOCP 기반 대규모 분산 MMORPG 게임 서버

개발 기간

3개월 · 1인 개발

언어 / 환경

C++20 · Windows · MSSQL

동시 접속

16,000 (자체 부하 테스트)

Repository

github.com/SJKIM99/MMORPG-with-IOCP

INDEX 목차

01	프로젝트 개요	성과
02	전체 아키텍처	구조
03	IOCP 네트워크 계층	구조
04	Zone 분산 — 처리량 4배	성능
05	Sector 시야 관리	성능
06	Lock-free 메모리 풀	성능
07	부하 테스트	검증

08	인벤토리 — 설계 원칙	신규
09	인벤토리 — 원자성 보장	신규
10	DB 쓰기 순서 — 레인 샤딩	신규 · 핵심
11	필드 드롭과 루팅 우선권	신규
12	코드 리뷰로 찾아낸 결함	신규
13	검증 방식 — Negative Control	신규
14	코드 구조 · 회고	정리

01 프로젝트 개요

수천 명이 동시에 접속하는 MMORPG 서버를 밑바닥부터 직접 설계했습니다. 단일 스레드로 처리하던 게임 로직을 월드 분할로 병렬화하고, 자체 제작한 부하 테스트 도구로 한계를 측정하며 개선했습니다.

16,000

동시 접속 (자체 부하 테스트)

4배

Zone 분산 적용 후 처리량

0

게임 로직의 락 (설계상 제거)

직접 설계한 것

- IOCP 기반 비동기 네트워크 계층
- 월드를 16개 Zone으로 나눈 스레드 분산 구조
- Lock-free 메모리 풀 (Windows SLIST)
- 인벤토리 · 아이템 · DB 영속화 파이프라인
- 봇 클라이언트 부하 테스트 도구 (시각화 포함)

기술 스택

Server	C++20, Windows IOCP, Worker Thread Pool
동시성	Zone 스레드 분산, Lock-free SLIST
Database	MSSQL, ODBC, 저장 프로시저
Client	SFML 2.5.1 (검증용 클라이언트)
Tooling	자체 제작 STRESS_TEST (OpenGL 대시보드)

02 전체 아키텍처

네트워크 I/O · 게임 로직 · DB를 **완전히 분리된 스레드 계층**으로 나눴습니다. 패킷은 반드시 그 플레이어를 소유한 Zone 스레드 위에서만 처리됩니다.



설계의 중심 원칙 — 한 플레이어의 상태는 오직 하나의 Zone 스레드만 건드린다. 그래서 게임 로직 전체에 **뮤텍스가 없고**, 인벤토리 같은 자료구조도 락 없이 안전합니다.

Timer Thread

회복 · 리스폰 · 아이템 소멸 등 예약 작업을 소유 Zone 스레드로 던져 실행합니다.

Zone Manager

좌표로 담당 Zone을 찾아 작업을 넘깁니다. Zone 경계를 넘을 때만 스레드 간 이관이 일어납니다.

DB Thread Pool

게임 스레드를 막지 않는 비동기 저장. 같은 플레이어의 쓰기는 순서가 보장됩니다 (10장).

03 IOCP 네트워크 계층

Windows IOCP로 수천 개의 소켓을 소수의 스레드가 처리합니다. 커널이 완료된 I/O만 알려주므로, 스레드가 소켓을 붙잡고 기다리는 시간이 없습니다.

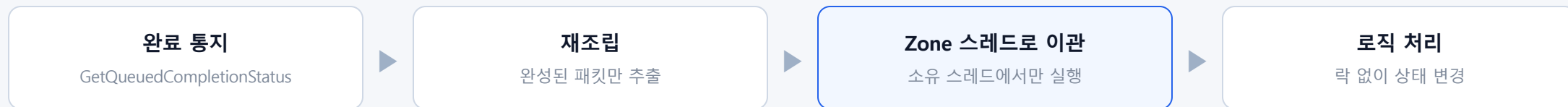
연결 수립

- **AcceptEx**로 접속 요청을 미리 등록해두고 대기
- 수락 즉시 IOCP에 소켓을 연결하고 `WSARecv` 예약
- `TCP_NODELAY`로 Nagle 알고리즘 해제 — 조작 반응 지연 제거

수신 처리

- TCP는 경계가 없으므로 **부분 수신**을 직접 재조립
- 패킷이 완성될 때까지 버퍼에 누적, 완성분만 처리
- 완성된 패킷은 **소유 Zone 스레드로 넘겨서 실행**

패킷 하나가 지나가는 길



결과 — 워커 스레드는 **I/O 대기**로 **멈추지 않고** 완료 통지만 처리합니다. 접속이 늘어도 스레드 수는 그대로라, 컨텍스트 스위칭 비용이 접속 수에 비례해 늘지 않습니다.

04 Zone 분산 — 처리량 4배

네트워크는 병렬인데 **게임 로직이 스레드 하나에 묶여 있어** 4,000명에서 한계에 부딪혔습니다. 월드를 4×4로 쪼개 Zone마다 전용 스레드를 붙였습니다.

BEFORE

GameLogicThread 1개가 전 월드를 직렬 처리

- 접속이 늘수록 큐가 밀림
- CPU 코어를 남기고도 **4,000명에서 지연 급증**



AFTER

월드를 16 Zone으로 분할, Zone마다 전용 스레드

- 서로 다른 Zone은 완전히 병렬
- **16,000명까지 안정적으로 처리**

Zone 경계는 어떻게 넘나

플레이어가 Zone 경계를 넘으면 **소유권을 다음 Zone 스레드로 이관**합니다. 이관이 끝나기 전까지는 이동·공격·인벤토리 패킷을 모두 무시해, 두 스레드가 같은 플레이어를 동시에 건드리는 순간이 존재하지 않습니다.

이 구조가 주는 것

- Zone 안에서는 **단일 스레드 = 락이 필요 없음**
- Zone 수를 늘리면 코어 수만큼 확장
- 이후 만든 인벤토리도 이 전제 위에서 락 없이 동작

05 Sector 시야 관리

"내 주변에 누가 보이는가"를 매번 **전체 오브젝트를 순회해서** 구하면 접속 수의 제곱으로 비용이 늘어납니다. Zone 내부를 다시 격자 (Sector)로 나눠 **인접한 9칸만** 확인합니다.

격자가 없다면

한 명이 움직일 때마다 월드의 모든 오브젝트와 거리 비교 → **접속 수 × 오브젝트 수**만큼의 연산. 1만 명이 동시에 움직이면 그 자체로 서버가 멈춥니다.

격자를 쓰면

내가 속한 칸과 인접 8칸, **총 9칸만 순회**. 전체 규모와 무관하게 **주변 밀도에만** 비용이 비례합니다.

시야 진입 / 이탈을 알리는 방법

이동 전후의 주변 목록을 비교해, 새로 들어온 대상에게는 **생성 알림**을, 빠져나간 대상에게는 **제거 알림**을 보냅니다. Zone 경계 바깥의 Sector는 배열에 아예 존재하지 않으므로, 별도 처리 없이 **Zone 경계가 자연스럽게 시야의 끝**이 됩니다.

06 Lock-free 메모리 풀

패킷 처리 컨텍스트를 매번 `new/delete` 하면, 수천 스레드가 힙 하나를 두고 경합합니다. 미리 만들어 둔 객체를 **돌려쓰는 풀**로 바꿨습니다.

문제 — ABA

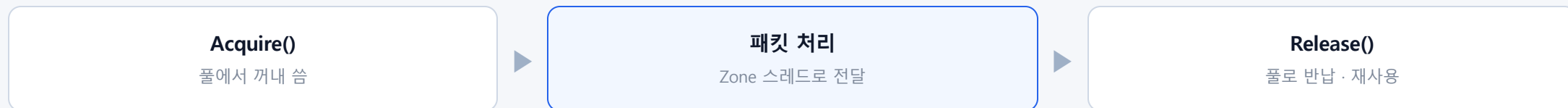
직접 lock-free 스택을 짜면 **ABA 문제**를 만납니다. A를 꺼내려는 순간 다른 스레드가 A를 꺼내 쓰고 되돌려 놓으면, 포인터 값은 같지만 내용은 달라져 **엉뚱한 메모리를 반환**합니다.

해결 — Windows SLIST

OS가 제공하는 **Interlocked Singly Linked List**를 freelist로 사용했습니다.

`InterlockedPushEntrySList` / `Pop` 이 **ABA를 구조적으로 차단**하고, 락 없이 안전합니다.

사용 흐름



07 부하 테스트

"몇 명까지 되는가"를 추측하지 않기 위해 부하 테스트 도구를 직접 만들었습니다. 봇이 실제 프로토콜로 접속해 이동·전투·인벤토리 행동을 수행하고, 지연이 임계치를 넘으면 자동으로 접속을 줄입니다.



봇이 실제로 하는 행동

- 이동 (0.4~0.6초) · 공격 (0.4~0.6초) · 스킬 (5초) · 채팅
- 바닥의 아이템을 향해 이동해서 줍기
- 체력이 60% 아래로 떨어지면 포션 사용
- 주운 장비 장착 / 해제, 가끔 버리기

측정 방식

이동 요청부터 서버 반영 통지까지의 왕복 지연을 상시 측정합니다. 지연이 기준을 넘으면 접속을 줄이고, 회복되면 다시 늘려 안정적으로 유지되는 최대 접속 수를 찾습니다.

PART 02

인벤토리 시스템

아이템을 얻고, 장착하고, 바닥에 떨어뜨리고, DB에 남긴다.

단순해 보이는 기능이지만 **동시성과 영속성이 만나는 지점**이라 가장 많은 설계 판단이 필요했습니다.

08 인벤토리 — 설계 원칙

"동작하는 인벤토리"가 아니라 **4만 명 규모에서도 버티는 인벤토리**를 목표로 했습니다. 그 과정에서 내린 세 가지 판단입니다.

① 아이템을 전역 맵에 넣지 않는다

모든 오브젝트를 담는 전역 관리자에 아이템까지 넣으면 **4만 명 × 30슬롯 = 120만 객체**가 들어가고, 장착·소비마다 **전역 락 경합**이 발생합니다.

인벤토리가 직접 소유하고, **바닥에 떨어진 아이템**만 예외로 등록합니다 — 남에게 보여야 하니까요.

② 슬롯은 배열이 아닌 맵

30칸을 통째로 잡는 대신 **슬롯번호 → 아이템** 맵을 씁니다. 빈 슬롯은 **키 자체가 없으므로** 메모리도, 순회 비용도 실제 보유량만큼만 듭니다.

대신 "없는 키를 만들 수 있다"는 위험이 생겼고, 이것이 실제 결함으로 이어졌습니다 (12장).

③ 변경 즉시 DB 반영

일정 주기로 몰아 저장하면 그 사이 서버가 죽었을 때 **아이템이 사라집니다**. 획득·장착·소비마다 곧바로 저장 요청을 보냅니다.

단, 게임 스레드는 막지 않고 **비동기로 넘기기만** 합니다. 이 선택이 다음 두 장의 과제를 만들었습니다.

세 판단의 공통점 — 세 가지 모두 **성능을 얻는 대신 새로운 위험을 떠안는** 선택이었습니다. 전역 등록을 피해 락 경합을 없앴 대신 필드 아이템은 예외 처리가 필요했고, 맵을 쓴 대신 **범위 검증 책임**이 생겼으며, 즉시 저장을 택한 대신 **쓰기 순서 문제**가 따라왔습니다. 다음 장부터는 그 대가를 어떻게 갚았는지 다룹니다.

09 인벤토리 — 원자성 보장

검을 장착한 상태에서 **다른 검을 장착하면** ①기존 해제 ②신규 장착, 두 가지가 함께 일어나야 합니다. 이 둘이 쪼개지면 무엇이 깨지는지부터 따졌습니다.

쪼개지면 생기는 일

- **메모리** — "둘 다 장착" 상태가 잠깐 관측되면 공격력 계산이 틀어짐
- **DB** — 두 슬롯을 따로 저장하는 사이 서버가 죽으면 **한쪽만 반영된 상태가 영구히** 남음

두 층 모두에서 보장

- **메모리** — Zone 스레드 단일 소유라 함수 하나가 끝날 때까지 **누구도 끼어들 수 없음**
- **DB** — 두 슬롯을 **하나의 트랜잭션**으로 저장하는 전용 프로시저를 만들어 처리

같은 원칙을 슬롯 교체에도 적용

슬롯 교체(A↔B)도 "논리적으로 한 동작"입니다. 따로 저장하면 아이템이 **두 슬롯 모두에 존재하거나 어디에도 없는** 상태가 DB에 남을 수 있어, 같은 트랜잭션 프로시저를 재사용했습니다.

기존에 이미 만들어 둔 기능이라도, 같은 종류의 결함이 있는지 함께 점검하고 고쳤습니다.

10 DB 쓰기 순서 — 레인 샤딩

트랜잭션을 걸었는데도 DB가 옛날 상태로 남는 경우가 있었습니다. 원인은 트랜잭션 안이 아니라, 요청이 DB에 도달하는 순서에 있었습니다.

문제 — 꺼낸 순서와 커밋된 순서가 다르다

DB 워커 8개가 공용 큐 하나를 나눠 가졌습니다. 큐에서 꺼내는 순서는 맞지만 꺼낸 뒤에는 각자 DB 왕복을 하므로, **먼저 꺼낸 것이 먼저 커밋된다는 보장이 없습니다**. 플레이어의 장착 → 해제를 빠르게 하면 "해제"가 먼저 커밋된 뒤 "장착"이 덮어쓸 수 있고, 그 상태로 서버가 죽으면 DB에는 장착된 채로 영구히 남습니다.

해결 — 같은 플레이어는 같은 레인

Lane 0 id 8 id 16 id 24

Lane 1 id 9 id 17

Lane 2 id 10 id 18

레인 8개, 스레드 하나가 레인 하나를 전담합니다. 아이템 저장은 `playerId % 8` 로 레인이 정해지므로 **한 플레이어의 쓰기는 넣은 순서 그대로** 처리됩니다.

왜 하필 `playerId % 8`

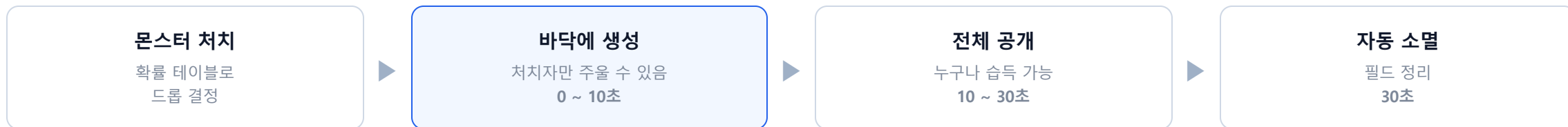
- **해시가 필요 없다** — playerId는 DB가 발급하는 auto-increment 순차 정수라, 나머지 연산만으로 **거의 완벽히 균등**하게 흩어집니다
- **병렬성 손실이 없다** — 스레드도 커넥션도 8개 그대로, 다른 플레이어끼리는 여전히 완전 병렬
- 줄어드는 건 "노는 워커가 아무거나 집어간다"는 유연성뿐인데, 이 쓰기는 **아무도 기다리지 않는 비동기 저장**이라 체감되지 않습니다

함께 고친 것 · 검토한 대안

- **교착 상태** — 같은 두 슬롯을 반대 방향으로 교체하면 서로의 행을 기다릴 수 있어, **항상 작은 슬롯 번호부터** 잠그도록 순서를 고정
- **대안** — 쓰기마다 버전 번호를 붙여 오래된 쓰기를 거부하는 방법도 있었지만, 스키마·프로시저를 모두 손대야 해서 **큐 구조만 바꾸면 되는** 레인 방식을 택함

11 필드 드롭과 루팅 우선권

처음엔 몬스터를 잡으면 아이템이 **인벤토리에 바로 꽂혔습니다**. 이를 **바닥에 떨어뜨리고 직접 줍는** 방식으로 바꾸면서, 소유권과 수명 문제가 따라왔습니다.



습득 경쟁은 왜 안 생기나

같은 칸에 서 있는 두 플레이어는 **반드시 같은 Zone**에 있고, Zone은 스레드 하나가 담당합니다. 두 요청은 **순서대로 실행될 수밖에 없어** 복제나 중복 습득이 구조적으로 불가능합니다.

실패해도 아이템이 사라지지 않게

"찾기 → 인벤토리에 넣기 → 성공했을 때만 필드에서 제거" 순서로 처리합니다. 인벤토리가 가득 차 실패하면 아이템은 바닥에 그대로 남고, 클라이언트에는 사유를 따로 알립니다.

수명 관리

우선권 만료(10초)와 소멸(30초)을 Timer Thread에 예약하고, 실행 시점에 **해당 Zone 스레드로 넘겨** 처리합니다. 먼저 습득되면 타이머는 대상이 없음을 확인하고 조용히 넘어갑니다.

바닥에 떨어진 아이템만 전역에 등록하는 이유

인벤토리 속 아이템은 주인만 알면 되지만, **바닥의 아이템은 남에게 보여야** 합니다. 그래서 아이템 클래스는 그대로 두고, **주인이 없는 상태일 때만** 전역 관리자와 시야 격자에 등록합니다. 필드 아이템은 동시에 존재하는 수가 인벤토리 아이템과 비교할 수 없이 적어, 8장에서 피하려던 **전역 락 경합 문제가 생기지 않습니다**.

PART 03

완성 이후, 스스로 점검하기

기능이 동작한 뒤에 코드 전체를 다시 읽으며 결함을 찾았습니다.

발견 → 원인 분석 → 수정 → 검증까지의 과정을 정리했습니다.

12 코드 리뷰로 찾아낸 결함

인벤토리를 완성한 뒤, "내가 짠 코드를 처음 보는 사람처럼" 다시 읽었습니다. 테스트는 통과하지만 특정 조건에서만 드러나는 결함 세 가지를 찾았습니다.

치명적

공격 쿨타임 우회

발견 — 공격 쿨타임을 **클라이언트가 보낸 시간 값**으로 갱신하고 있었습니다. 조작된 클라이언트가 매번 0 을 보내면 검사가 항상 통과해 쿨타임이 사실상 사라집니다.

수정 — 패킷에서 시간 필드를 **아예 제거**하고 서버 시계만 신뢰하도록 통일했습니다. 클라이언트가 보낼 수 없으면 위조할 수도 없습니다.

데이터 유실

DB 커밋 순서 역전

발견 — 워커 8개가 공용 큐를 나눠 가지면서 같은 플레이어의 쓰기 순서가 **뒤바뀔 수 있었습니다**.

수정 — 플레이어별 레인 고정 (10장).

함께 발견한 **교착 상태**도 처리했습니다. 같은 두 슬롯을 반대 방향으로 교체하면 서로의 행을 기다릴 수 있어, **항상 작은 슬롯 번호부터 잠그도록** 순서를 고정했습니다.

아이템 소실

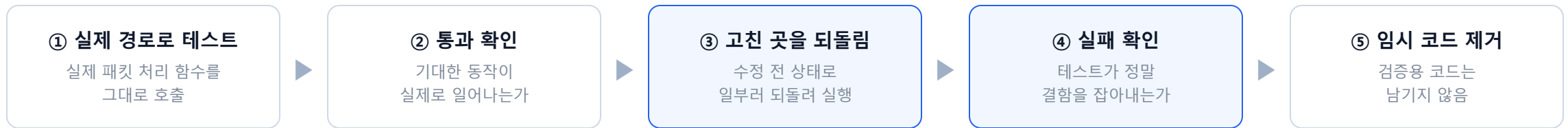
슬롯 범위 미검증

발견 — 대부분의 함수는 "없는 슬롯이면 실패"라 안전했지만, 슬롯 교체만 맵에 **없던 키를 새로 만들 수 있었습니다**. 범위 밖 번호를 보내면 아이템이 화면에도 안 보이고 다시 꺼낼 수도 없는 곳으로 영구히 사라집니다.

수정 — 검증을 패킷 처리부가 아닌 **인벤토리 내부**에 뒀습니다. 모든 경로가 지나는 길목이라 한 곳만 막으면 전부 덮입니다.

13 검증 방식 — Negative Control

수정한 뒤 테스트가 통과하면 끝일까요? **"통과했다"보다 "실패할 수는 있는가"를 먼저 확인**했습니다. 아무것도 검사하지 않는 테스트도 언제나 통과하기 때문입니다.



사례 1 — DB 레인

"모든 레인에 담당 스레드가 있는가"를 확인하는 테스트를 만들고, **일부러 레인 하나를 띄우지 않고** 실행했습니다.

```
// 정상
[PASS] all lanes drained    16 / 16

// 레인 하나를 뺀 경우
[FAIL] all lanes drained    14 / 16
```

그 레인으로 배정된 **2건만 정확히** 처리되지 않았습니다. 테스트가 실제로 이 결함을 잡아낸다는 확인입니다.

사례 2 — 슬롯 범위 검증

범위 밖 슬롯 교체를 막는 검사를 **잠시 비활성화**하고 같은 테스트를 돌렸습니다.

```
// 검사를 끈 경우
[FAIL] 범위 밖 슬롯 교체가 거부되는가
[FAIL] 원래 아이템이 제 자리에 있는가
[FAIL] 이후 정상 교체가 동작하는가
```

아이템이 실제로 **접근 불가능한 슬롯으로 넘어가**, 이후 정상 동작 테스트까지 연쇄로 실패했습니다. **"영구히 사라진다"가 말이 아니라 실제 동작임을 확인**한 셈입니다.

14 코드 구조

역할이 다른 코드는 폴더부터 분리했습니다. 새 기능을 넣을 때 어디를 열어야 하는지가 구조만 보고 드러나는 것을 목표로 했습니다.

모듈	역할
Network	IOCP, 소켓, 세션, 패킷 송수신
Thread	워커 · 게임로직 · 타이머 · DB 스레드
Zone	월드 분할, Zone 소유권 이관
Area	Sector 격자, 시야 계산
Item	아이템 · 인벤토리 · 드롭 · 필드 아이템
User / Monster	플레이어 · 몬스터 행동
Route	패킷 종류별 진입점
Core	DB 연결 · 커넥션 풀 · 공용 유틸

프로토콜은 단 하나의 원본만

서버 · 클라이언트 · 부하 테스트 도구가 각자 프로토콜 정의를 복사해 들고 있으면, 하나가 바뀔 때 나머지가 조용히 어긋납니다. 실제로 복사본에서 패킷 순서와 필드가 어긋나 있던 것을 발견하고, 서버 정의 하나만 참조하도록 통합했습니다.

주석은 "무엇"이 아니라 "왜"

코드를 보면 아는 내용 대신, 왜 이 방식을 택했는지와 다르게 하면 무엇이 깨지는지를 남겼습니다. 몇 달 뒤의 제가 같은 판단을 다시 하지 않도록.

15 회고

이 프로젝트에서 가장 크게 남은 것은 특정 기술이 아니라, "동작한다"와 "안전하다" 사이의 거리를 재는 감각이었습니다.

병목은 재봐야 안다

네트워크를 아무리 최적화해도 로직 스레드가 하나면 소용없었습니다. **부하 테스트 도구를 직접 만들어** 숫자로 확인한 뒤에야 Zone 분산이라는 답이 나왔습니다.

동시성은 구조로 푼다

락을 촘촘히 거는 대신 "한 플레이어는 한 스레드만 건드린다"는 규칙을 세웠습니다. 규칙이 서니 인벤토리처럼 복잡한 기능도 락 없이 안전하게 엮을 수 있었습니다.

버그는 재현되기 전에 찾는다

쿨타임 우회도, 커밋 순서 역전도 **재현된 적 없는 결함**이었습니다. 코드를 다시 읽는 시간을 따로 확보한 덕분에 운영에서 만나기 전에 처리할 수 있었습니다.

다음에 이어서 하고 싶은 것

- Zone 간 부하 편차를 감지해 **동적으로 경계를 조정**하기
- DB 쓰기량이 한계에 닿으면 **버전 기반 쓰기**로 전환 (10장의 대안)
- 거래 · 창고처럼 **두 플레이어의 상태가 동시에 바뀌는** 기능 설계